

Leetcode 105. Construct Binary Tree from Preorder and Inorder Traversal

Achyutha Duddebanda

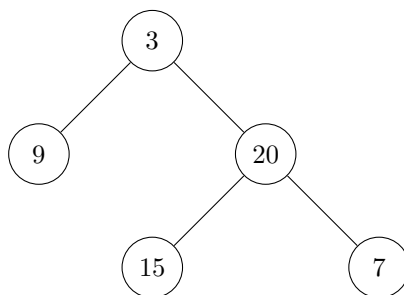
May 6th 2026

Problem

We are given 2 integer arrays that store the preorder and inorder traversals of a binary tree. Our goal is to reconstruct the original tree given this information.

Example

Consider the following binary tree:



Preorder traversal (Root → Left → Right):

[3, 9, 20, 15, 7]

Inorder traversal (Left → Root → Right):

[9, 3, 15, 20, 7]

Key Observation

The preorder traversal always gives the root first.

`preorder[0] = 3`

Using the inorder traversal:

$[9, \boxed{3}, 15, 20, 7]$

everything to the left of 3 belongs to the left subtree, and everything to the right belongs to the right subtree.

Thus:

Left subtree inorder = $[9]$

Right subtree inorder = $[15, 20, 7]$

We then recursively repeat this process for each subtree.

Solution

Initially, I understood the main divide-and-conquer strategy:

- Use preorder traversal to determine the root.
- Use inorder traversal to split the tree into left and right subtrees.
- Recursively build both subtrees.

However, I struggled to translate this idea clearly into code.

What Went Wrong

I introduced many unnecessary conditions and extra logic because I was not fully trusting the recursive structure of the problem.

Some mistakes I made:

- Trying to manually manage too many indices.
- Adding complicated base cases.
- Overthinking how the preorder array should advance recursively.
- Attempting to explicitly control subtree construction instead of letting recursion naturally handle it.

After reviewing the official solution, I realized that my overall approach was already mostly correct. The cleaner implementation relied on a very simple idea:

The next unused preorder element is always the root of the current subtree.

This means that a single index pointer, i , can be updated after each root has been processed and added to the tree. This is because the left subtree(smaller values of i) will get processed first before the right subtree is examined.

Core Recursive Structure

For each recursive call:

1. Take the next preorder element as the root.
2. Find that value in the inorder array.
3. Everything left of the value belongs to the left subtree.
4. Everything right of the value belongs to the right subtree.
5. Recursively build the left subtree first, then the right subtree.

The recursive ordering naturally works because preorder traversal follows:

Root $\rightarrow$ Left $\rightarrow$ Right

Tips

- Focus on the recursive definition instead of trying to manage every detail manually.
- Trust recursion to construct the subtrees naturally.
- Keep recursive parameters minimal. Usually only the inorder boundaries and a preorder index are needed.
- After solving a recursion problem, try rewriting the solution from scratch later without looking at notes. This helps reinforce the underlying pattern.